

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Kavinsoorya K S
Register Number	3122247001030
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 30, 2026

1 Objective

The objective of this experiment is to study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the classification performance of ten machine learning models. Each model is trained and evaluated under two settings: using the original standardized features (**No-PCA**) and using PCA-reduced features (**With-PCA**). Hyperparameter tuning via GridSearchCV with 5-fold stratified cross-validation is performed for all models under both conditions. The experiment compares accuracy, precision, recall, F1-score, and ROC-AUC to determine which models benefit from dimensionality reduction and which do not.

2 Problem Statement

Given a diagnostic dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{569}$ where $\mathbf{x}_i \in \mathbb{R}^{30}$ represents continuous features computed from cell nuclei measurements in breast mass biopsies and $y_i \in \{0, 1\}$ denotes the diagnostic label (0 for Malignant, 1 for Benign), the task is to:

1. Standardize the feature space and apply PCA to obtain a reduced representation $\mathbf{x}'_i \in \mathbb{R}^k$ where $k < 30$, retaining at least 95% of the total variance.
2. Train ten classifiers (SVM, Naïve Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and Stacking) on both the original and PCA-reduced feature spaces.
3. Perform hyperparameter tuning and 5-fold cross-validation for each model under both settings.
4. Compare and analyze whether PCA improved accuracy, F1-score, or prediction stability.

3 Dataset Description

The analysis uses the **Wisconsin Diagnostic Breast Cancer (WDBC)** dataset from the scikit-learn library. Table 1 summarizes the dataset characteristics.

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC) Dataset
Dataset Source	scikit-learn (<code>sklearn.datasets.load_breast_cancer</code>)
Number of Samples	569

Number of Features	30 continuous numerical attributes
Number of Classes	2 (Malignant: 0, Benign: 1)
Missing Values	0 (complete dataset)
Duplicate Rows	0
Train-Test Split	80% Training (455) / 20% Testing (114)
Training Class Distribution	Benign (1): 285 (62.64%), Malignant (0): 170 (37.36%)
Testing Class Distribution	Benign (1): 72 (63.16%), Malignant (0): 42 (36.84%)

The 30 features describe ten physical properties of cell nuclei (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension), each summarized by three statistics: mean, standard error, and worst (mean of the three largest values).

4 Exploratory Data Analysis

4.1 Class Distribution

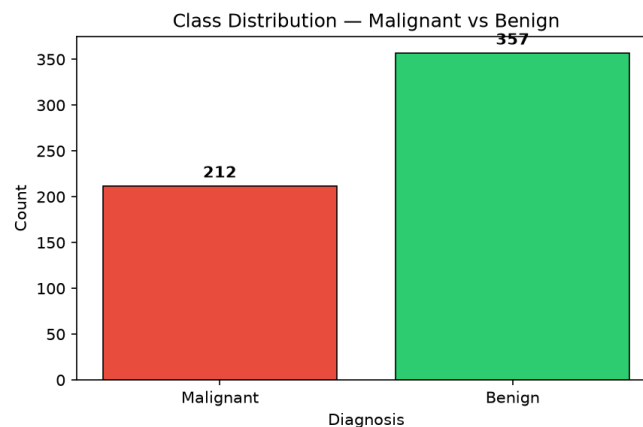


Figure 1: Class Distribution of Malignant (0) vs. Benign (1) Samples

Inference: The dataset contains 357 benign samples (62.74%) and 212 malignant samples (37.26%), indicating moderate class imbalance. Stratified sampling was used during the train-test split to preserve this distribution in both the training set (285 Benign, 170 Malignant) and testing set (72 Benign, 42 Malignant). This prevents models from developing bias toward the majority class.

4.2 Feature Correlation Analysis

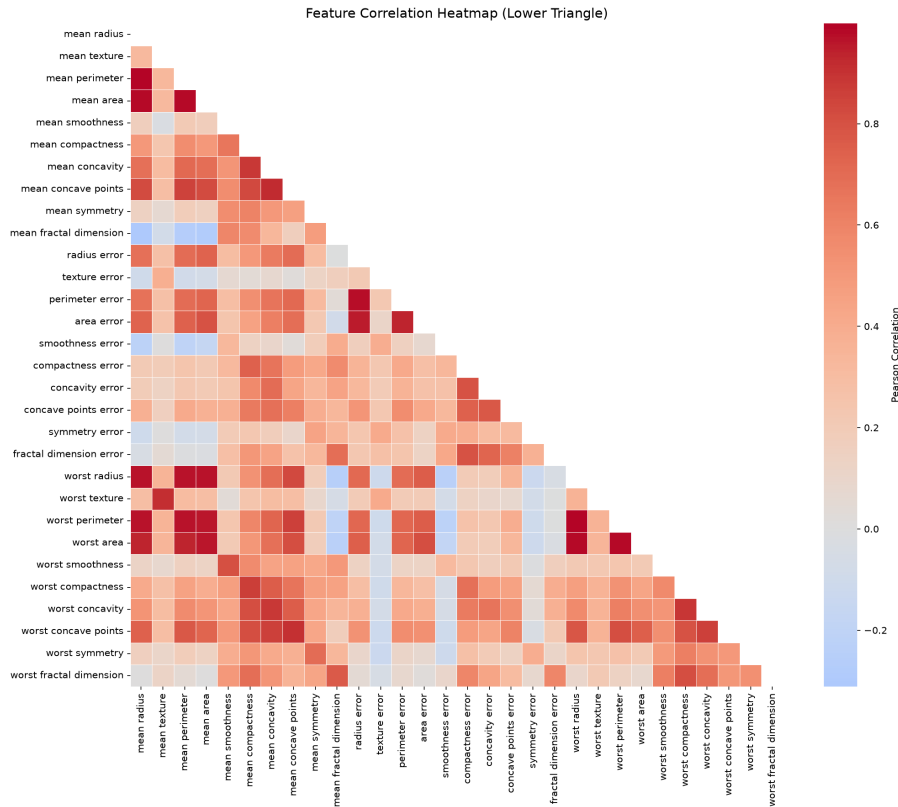


Figure 2: Pairwise Correlation Heatmap of 30 Features (Lower Triangle)

Inference: The heatmap reveals high multicollinearity among several feature groups, particularly radius, perimeter, and area variants (Pearson coefficients approaching 1.0). This strong redundancy across features makes the dataset well-suited for PCA, which can compress correlated variables into fewer orthogonal components. For linear models like Logistic Regression, multicollinearity can cause coefficient instability, so PCA's decorrelation effect is expected to benefit them.

5 Data Preprocessing

The preprocessing pipeline consists of:

1. **Feature-Target Separation:** The 30 numerical features were separated from the binary target variable.
2. **Train-Test Split:** An 80-20 stratified split was applied using `random.state=42` to ensure reproducibility and class balance.
3. **Standardization:** `StandardScaler` was fitted on the training set and applied to both training and testing sets to prevent data leakage. After scaling, training features have zero mean and unit variance.
4. **PCA:** Applied to the standardized training features (details in the PCA Design section).

Listing 1: Preprocessing and PCA Pipeline

```

1 scaler = StandardScaler()
2 X_train_scaled = scaler.fit_transform(X_train)
3 X_test_scaled = scaler.transform(X_test)
4
5 pca = PCA(n_components=0.95, random_state=42)
6 X_train_pca = pca.fit_transform(X_train_scaled)
7 X_test_pca = pca.transform(X_test_scaled)

```

6 PCA Design

PCA was applied with a 95% variance retention threshold, fitted only on the training data. This resulted in **10 principal components** (out of 30 original features) capturing **95.27%** of the total variance.

Table 2: PCA Variance Summary

Setting	Components	Variance (%)	Justification
With-PCA	10 (95% target)	95.27%	Retain maximum information with minimum dimensions

Table 3: Component-wise Explained Variance

Component	Individual (%)	Cumulative (%)
PC1	44.41	44.41
PC2	18.94	63.36
PC3	9.54	72.90
PC4	6.72	79.63
PC5	5.52	85.14
PC6	3.93	89.08
PC7	2.18	91.26
PC8	1.58	92.84
PC9	1.28	94.12
PC10	1.15	95.27

6.1 Scree Plot

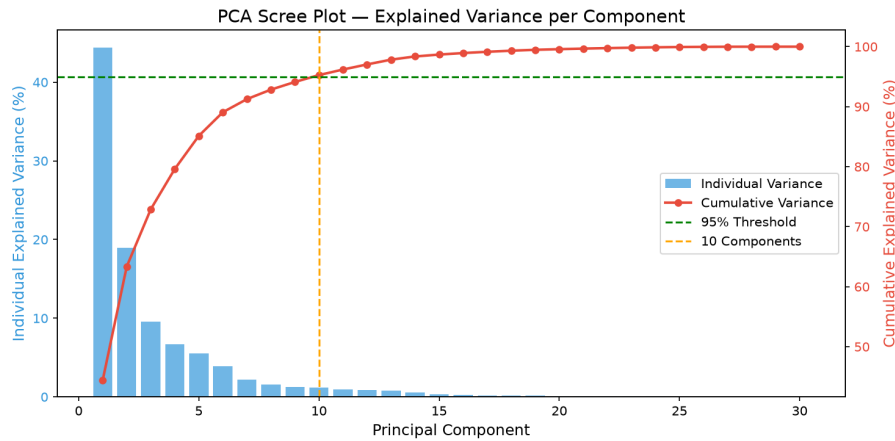


Figure 3: PCA Scree Plot — Individual and Cumulative Explained Variance

Inference: The first principal component alone captures 44.41% of the variance, and the first three components account for 72.90%. The curve flattens after approximately 10 components, confirming that the remaining 20 components contribute minimal information. The 95% threshold line intersects the cumulative curve at 10 components, validating the chosen retention target.

7 Machine Learning Algorithm

7.1 Principal Component Analysis (PCA)

Working Principle: PCA is a linear dimensionality reduction technique that transforms the original feature space into a new set of orthogonal axes (principal components) ordered by the amount of variance they capture. By projecting the data onto the top k components, PCA reduces dimensionality while preserving the maximum possible variance.

Mathematical Formulation: Given a centered data matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$, PCA computes the covariance matrix $\mathbf{C} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$ and performs eigendecomposition $\mathbf{C} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$, where $\mathbf{V}$ contains the eigenvectors (principal directions) and $\mathbf{\Lambda}$ contains the eigenvalues (variance along each direction). The reduced representation is $\mathbf{Z} = \mathbf{X} \mathbf{V}_k$, where $\mathbf{V}_k \in \mathbb{R}^{p \times k}$ contains the top k eigenvectors.

Advantages: Removes multicollinearity, reduces noise, lowers computational cost.

Limitations: Linear transformation only, components lose feature interpretability.

7.2 Classifiers

The following ten classifiers were evaluated:

1. **SVM:** Maximum-margin hyperplane classifier with kernel transformations.
2. **Naïve Bayes:** Probabilistic classifier using Bayes' theorem with feature independence assumption.

3. **KNN**: Instance-based learner classifying by majority vote among k nearest neighbors.
4. **Logistic Regression**: Linear model using sigmoid function for probabilistic classification.
5. **Decision Tree**: Recursive axis-aligned partitioning of the feature space.
6. **Random Forest**: Ensemble of decision trees trained on bootstrap samples with random feature subsets.
7. **AdaBoost**: Sequential boosting with weighted weak learners focusing on misclassified samples.
8. **Gradient Boosting**: Sequential boosting minimizing a differentiable loss function via pseudo-residuals.
9. **XGBoost**: Regularized gradient boosting with built-in L1/L2 regularization and efficient implementation.
10. **Stacking**: Heterogeneous ensemble combining diverse base learners through a meta-learner.

8 Experimental Setup

Table 4: Experimental Setup Specifications

Parameter	Specification
Python Version	3.13
Scikit-Learn Version	1.9.0
XGBoost Version	3.0+
IDE	Jupyter Notebook
Operating System	macOS Sequoia
Hardware	Apple Silicon M-Series / 16 GB Unified Memory

9 Hyperparameter Selection

Hyperparameter tuning was performed for all 10 models under both No-PCA and With-PCA settings using GridSearchCV with 5-fold stratified cross-validation.

9.1 SVM

Kernel	C	Gamma	Perf. (No-PCA)	Perf. (With-PCA)
linear	0.1	scale	0.9758	0.9714
linear	0.1	auto	0.9758	0.9714
linear	1	scale	0.9714	0.9780

Kernel	C	Gamma	Perf. (No-PCA)	Perf. (With-PCA)
linear	1	auto	0.9714	0.9780
linear	10	scale	0.9670	0.9736
linear	10	auto	0.9670	0.9736
rbf	0.1	scale	0.9582	0.9582
rbf	0.1	auto	0.9209	0.9560
rbf	1	scale	0.9758	0.9758
rbf	1	auto	0.9560	0.9736
rbf	10	scale	0.9714	0.9692
rbf	10	auto	0.9604	0.9670

Table 5: SVM — Hyperparameter Tuning Results

Best Parameters (No-PCA): kernel=linear, C=0.1, gamma=scale (CV Accuracy: 97.58%)

Best Parameters (With-PCA): kernel=linear, C=1, gamma=scale (CV Accuracy: 97.80%)

9.2 Naïve Bayes

Smoothing Parameter	Perf. (No-PCA)	Perf. (With-PCA)
1e-11	0.9341	0.9165
1e-10	0.9341	0.9165
1e-09	0.9341	0.9165
1e-08	0.9297	0.9165
1e-07	0.9297	0.9165
1e-06	0.9297	0.9143
1e-05	0.9275	0.9077

Table 6: Naïve Bayes — Smoothing Choices

Best Parameters (No-PCA): var_smoothing=1e-11 (CV Accuracy: 93.41%)

Best Parameters (With-PCA): var_smoothing=1e-11 (CV Accuracy: 91.65%)

9.3 KNN

k	Weights	Metric	Perf. (No-PCA)	Perf. (With-PCA)
3	uniform	manhattan	0.9714	0.9648
5	uniform	euclidean	0.9670	0.9648
5	uniform	manhattan	0.9670	0.9560
3	uniform	euclidean	0.9648	0.9604
3	distance	manhattan	0.9648	0.9604

Table 7: KNN — Hyperparameter Tuning (Top 5)

Best Parameters (No-PCA): k=3, weights=uniform, metric=manhattan (CV Accuracy: 97.14%)

Best Parameters (With-PCA): k=5, weights=uniform, metric=euclidean (CV Accuracy: 96.48%)

9.4 Logistic Regression

C	Penalty	Solver	Perf. (No-PCA)	Perf. (With-PCA)
0.01	l2	lbfgs	0.9780	0.9802
0.1	l2	lbfgs	0.9824	0.9824
1	l2	lbfgs	0.9802	0.9802
10	l2	lbfgs	0.9736	0.9802
100	l2	lbfgs	0.9736	0.9780

Table 8: Logistic Regression — Hyperparameter Tuning

Best Parameters (No-PCA): C=0.1 (CV Accuracy: 98.24%)

Best Parameters (With-PCA): C=0.1 (CV Accuracy: 98.24%)

9.5 Decision Tree

Max Depth	Min Split	Criterion	Perf. (No-PCA)	Perf. (With-PCA)
3	2	entropy	0.9319	0.9297
5	10	entropy	0.9297	0.9385
3	5	entropy	0.9297	0.9297
5	2	gini	0.9275	0.9275
7	5	gini	0.9275	0.9275

Table 9: Decision Tree — Hyperparameter Tuning (Top 5)

Best Parameters (No-PCA): criterion=entropy, max_depth=3, min_samples_split=2 (CV Accuracy: 93.19%)

Best Parameters (With-PCA): criterion=entropy, max_depth=5, min_samples_split=10 (CV Accuracy: 93.85%)

9.6 Random Forest

N Est.	Max Depth	Min Split	Perf. (No-PCA)	Perf. (With-PCA)
200	5	5	0.9626	0.9560
100	5	5	0.9626	0.9538
50	5	5	0.9604	0.9516
200	10	2	0.9604	0.9582
200	None	2	0.9604	0.9604

Table 10: Random Forest — Hyperparameter Tuning (Top 5)

Best Parameters (No-PCA): n_estimators=200, max_depth=5, min_split=5 (CV Accuracy: 96.26%)

Best Parameters (With-PCA): n_estimators=200, max_depth=None, min_split=2 (CV Accuracy: 96.04%)

9.7 AdaBoost

N Est.	LR	Depth	Perf. (No-PCA)	Perf. (With-PCA)
100	1.0	1	0.9802	0.9670
50	1.0	1	0.9780	0.9626
200	1.0	1	0.9780	0.9692
200	0.5	1	0.9758	0.9626
200	0.5	3	0.9714	0.9714

Table 11: AdaBoost — Hyperparameter Tuning (Top 5)

Best Parameters (No-PCA): n_estimators=100, learning_rate=1.0, max_depth=1 (CV Accuracy: 98.02%)

Best Parameters (With-PCA): n_estimators=200, learning_rate=0.5, max_depth=3 (CV Accuracy: 97.14%)

9.8 Gradient Boosting

N Est.	LR	Depth	Perf. (No-PCA)	Perf. (With-PCA)
200	0.2	3	0.9736	0.9626
100	0.2	3	0.9714	0.9604
200	0.1	3	0.9692	0.9626
100	0.1	3	0.9670	0.9604
200	0.2	5	0.9670	0.9604

Table 12: Gradient Boosting — Hyperparameter Tuning (Top 5)

Best Parameters (No-PCA): n_estimators=200, learning_rate=0.2, max_depth=3 (CV Accuracy: 97.36%)

Best Parameters (With-PCA): n_estimators=200, learning_rate=0.1, max_depth=3 (CV Accuracy: 96.26%)

9.9 XGBoost

N Est.	LR	Depth	Sub.	Perf. (NP)	Perf. (WP)
100	0.1	3	1.0	0.9714	0.9648
100	0.1	3	0.8	0.9714	0.9604
200	0.1	3	1.0	0.9714	0.9626
200	0.1	3	0.8	0.9692	0.9670

N Est.	LR	Depth	Sub.	Perf. (NP)	Perf. (WP)
100	0.2	3	1.0	0.9692	0.9516

Table 13: XGBoost — Hyperparameter Tuning (Top 5)

Best Parameters (No-PCA): n_estimators=100, learning_rate=0.1, max_depth=3, subsample=1.0 (CV Accuracy: 97.14%)

Best Parameters (With-PCA): n_estimators=200, learning_rate=0.1, max_depth=5, subsample=0.8 (CV Accuracy: 96.70%)

9.10 Stacking

Base Models	Meta Learner	CV Acc (NP)	CV Acc (WP)
SVM, Naïve Bayes, KNN, Decision Tree, Random Forest	Logistic Regression	97.14%	97.58%

Table 14: Stacking Classifier — Configuration and CV Performance

The best hyperparameters from each model’s individual tuning were used for the Stacking base estimators. The meta-learner (Logistic Regression, max_iter=5000) combines the out-of-fold predictions of the five base classifiers.

10 Results

10.1 5-Fold Cross-Validation (No-PCA vs With-PCA)

Model	F1	F2	F3	F4	F5	Avg (NP)	Avg (WP)
SVM	0.9560	0.9780	0.9780	0.9780	0.9890	0.9758	0.9780
Naïve Bayes	0.9121	0.9670	0.8901	0.9560	0.9451	0.9341	0.9165
KNN	0.9780	0.9890	0.9560	0.9560	0.9780	0.9714	0.9648
Log. Regression	0.9780	0.9780	0.9890	0.9780	0.9890	0.9824	0.9824
Decision Tree	0.9011	0.9451	0.9121	0.9341	0.9670	0.9319	0.9385
Random Forest	0.9560	0.9670	0.9341	0.9670	0.9890	0.9626	0.9604
AdaBoost	0.9890	1.0000	0.9451	0.9780	0.9890	0.9802	0.9714
Grad. Boosting	0.9780	0.9890	0.9560	0.9560	0.9890	0.9736	0.9626
XGBoost	0.9890	0.9780	0.9451	0.9560	0.9890	0.9714	0.9670
Stacking	0.9670	0.9780	0.9451	0.9780	0.9890	0.9714	0.9758

Table 15: 5-Fold Cross-Validation Accuracy (No-PCA Fold Scores Shown)

10.2 Test Performance Comparison

Table 16: Test Performance — No-PCA vs With-PCA

Model	No-PCA					With-PCA				
	Acc	Prec	Rec	F1	AUC	Acc	Prec	Rec	F1	AUC
SVM	.9825	.9861	.9861	.9861	.9937	.9649	.9857	.9583	.9718	.9950
Naïve Bayes	.9298	.9444	.9444	.9444	.9868	.9211	.9315	.9444	.9379	.9709
KNN	.9649	.9595	.9861	.9726	.9714	.9561	.9589	.9722	.9655	.9788
Log. Reg.	.9737	.9726	.9861	.9793	.9957	.9737	.9726	.9861	.9793	.9954
Dec. Tree	.9474	.9459	.9722	.9589	.9448	.9298	.9444	.9444	.9444	.9426
Rand. Forest	.9561	.9589	.9722	.9655	.9927	.9386	.9577	.9444	.9510	.9864
AdaBoost	.9561	.9467	.9861	.9660	.9818	.9649	.9722	.9722	.9722	.9931
Grad. Boost	.9561	.9467	.9861	.9660	.9957	.9561	.9467	.9861	.9660	.9878
XGBoost	.9474	.9459	.9722	.9589	.9934	.9474	.9583	.9583	.9583	.9911
Stacking	.9649	.9595	.9861	.9726	.9940	.9561	.9718	.9583	.9650	.9911

Best Model (No-PCA): SVM — Accuracy: 98.25%, F1: 0.9861, AUC: 0.9937

Best Model (With-PCA): Logistic Regression — Accuracy: 97.37%, F1: 0.9793, AUC: 0.9954

11 Result Visualization

11.1 Confusion Matrices — No-PCA

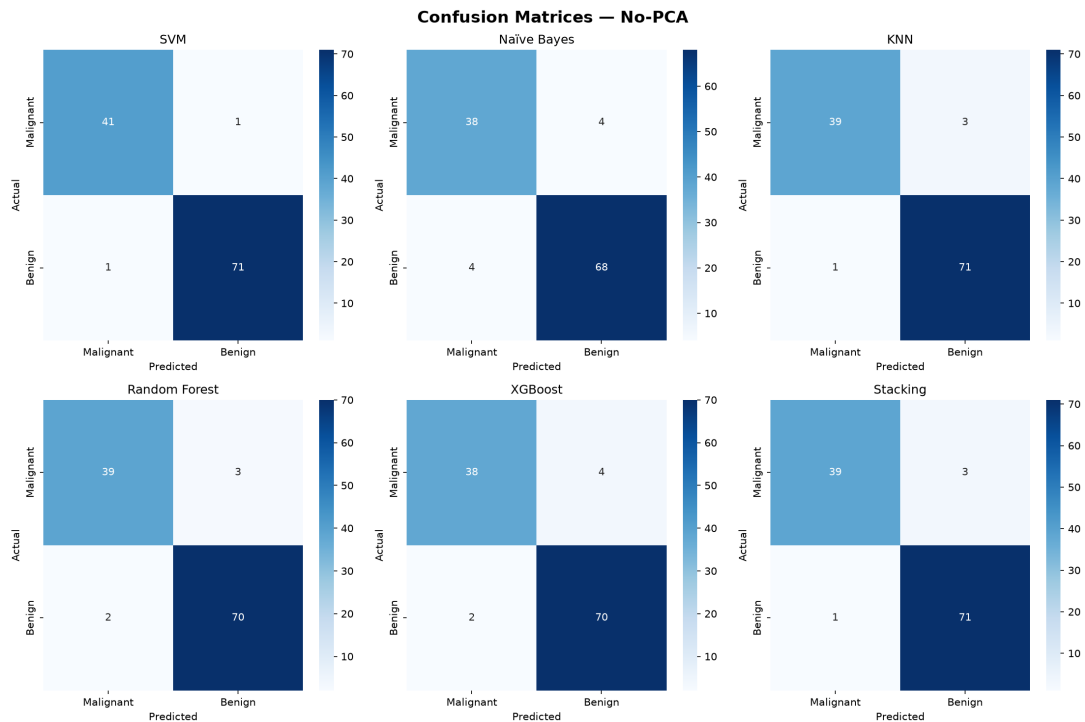


Figure 4: Confusion Matrices for Selected Models (No-PCA)

Inference: The No-PCA confusion matrices show that SVM, KNN, and Stacking achieve very few misclassifications on the test set. SVM records the fewest total errors, with most models achieving high true positive rates for the benign class. The pattern of false negatives and false positives varies across model types, reflecting their different decision boundary characteristics.

11.2 Confusion Matrices — With-PCA

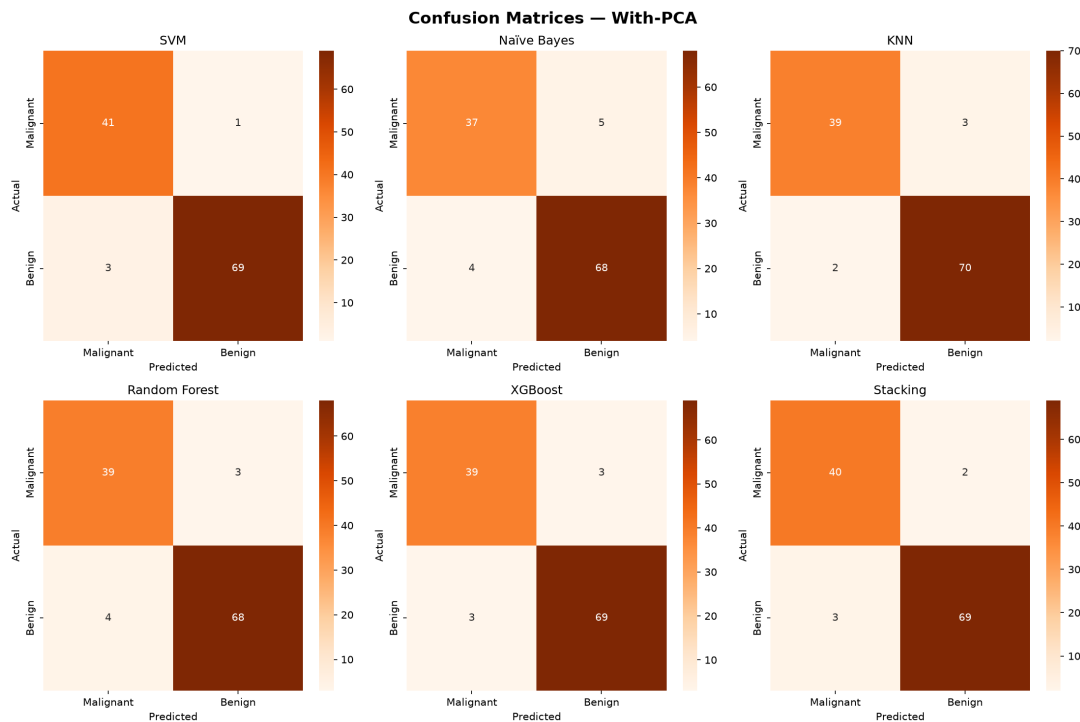


Figure 5: Confusion Matrices for Selected Models (With-PCA)

Inference: After PCA, most models show a slight increase in misclassifications compared to their No-PCA counterparts, consistent with the small information loss from dimensionality reduction. However, AdaBoost shows improvement with PCA, suggesting that the reduced feature space helped its sequential learning process focus on more informative directions.

11.3 ROC Curves

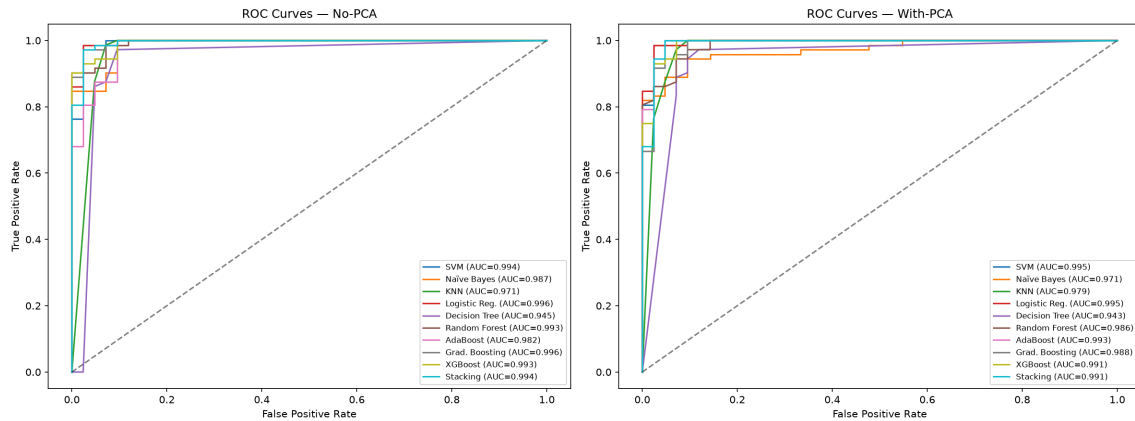


Figure 6: ROC Curve Comparison — No-PCA (Left) vs With-PCA (Right)

Inference: The ROC curves demonstrate that all models achieve high AUC values (> 0.94) under both settings. Logistic Regression and Gradient Boosting consistently maintain the highest AUC values. The With-PCA curves are slightly closer together, indicating that PCA narrows the performance gap between models by providing a more standardized feature representation.

11.4 Precision-Recall Curves

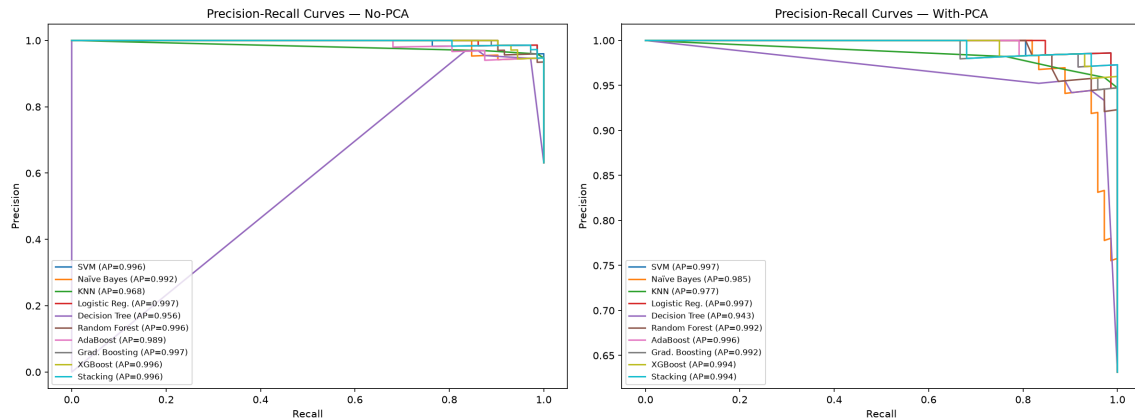


Figure 7: Precision-Recall Curves — No-PCA (Left) vs With-PCA (Right)

Inference: The Precision-Recall curves show that most models maintain high precision across a wide range of recall thresholds. Under No-PCA, Logistic Regression and SVM show the best precision-recall trade-offs. With PCA, the curves remain comparable, confirming that the dimensionality reduction preserves the discriminative signal for most classifiers.

11.5 Model Comparison — Accuracy and F1

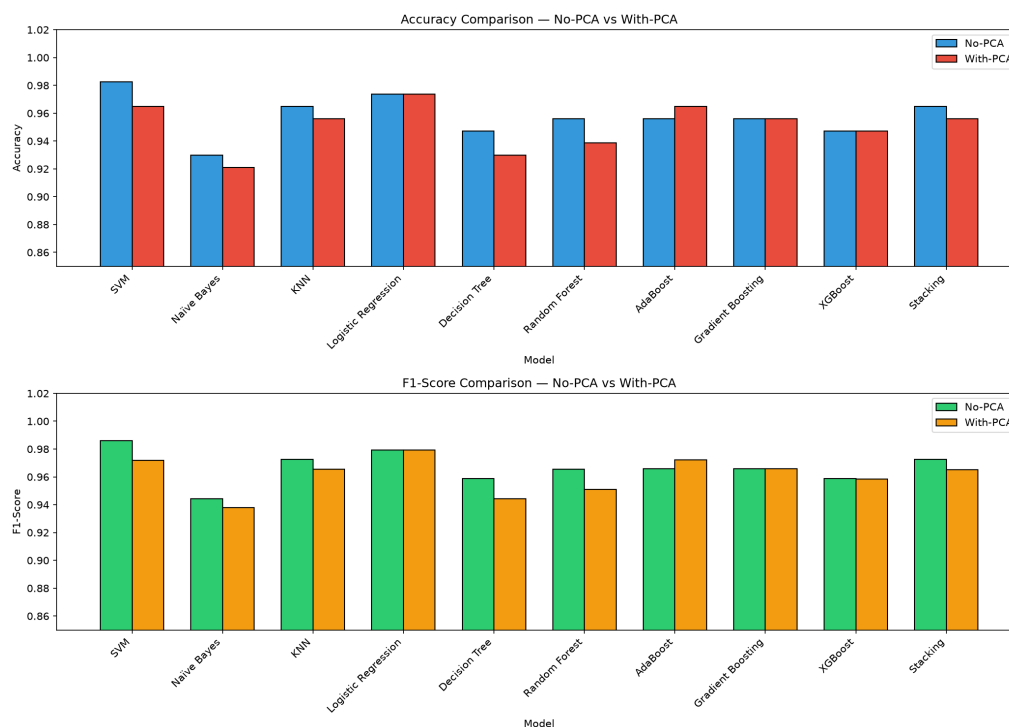


Figure 8: Accuracy and F1-Score Comparison — No-PCA vs With-PCA

Inference: The grouped bar charts show that most models achieve similar performance under both settings, with differences typically within 1-2 percentage points. AdaBoost is the only model that noticeably improved with PCA on this dataset, while SVM showed the largest decrease. Logistic Regression and Gradient Boosting remained unchanged, demonstrating robustness to the dimensionality reduction.

11.6 PCA Impact Heatmap

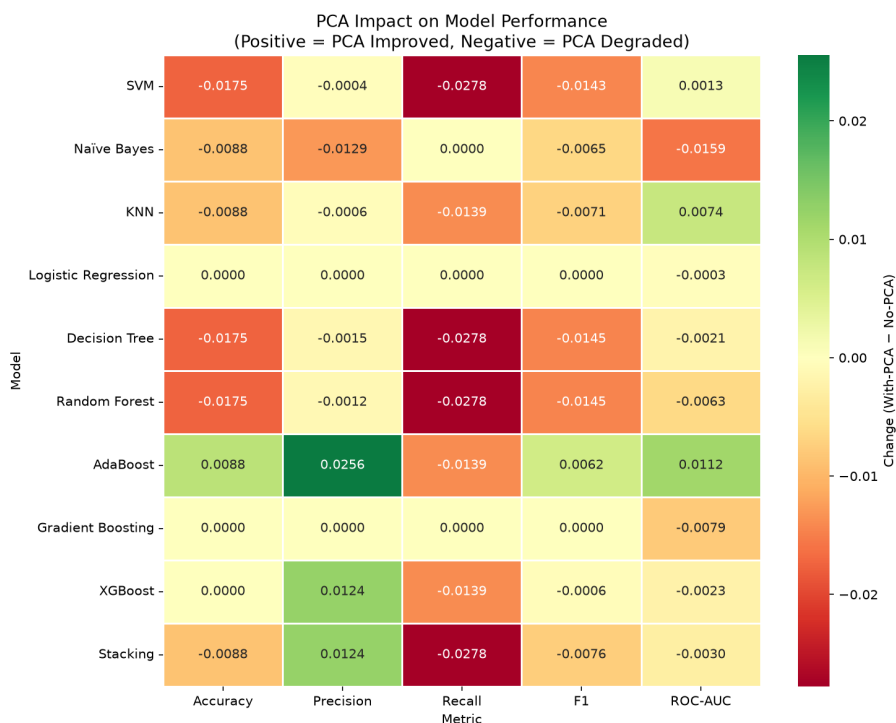


Figure 9: PCA Impact on Model Performance (With-PCA – No-PCA)

Inference: The heatmap quantifies the exact effect of PCA on each model-metric combination. Green cells indicate improvement, red cells indicate degradation. Logistic Regression and Gradient Boosting show near-zero changes across all metrics, while SVM and Decision Tree show consistent decreases. AdaBoost shows improvements in accuracy, precision, and AUC, suggesting that PCA removed noisy features that were interfering with its sequential learning.

11.7 Cross-Validation Stability

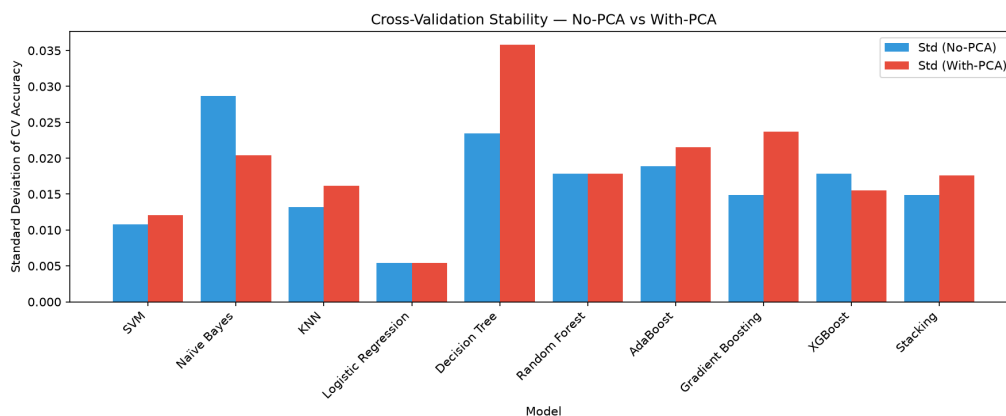


Figure 10: Cross-Validation Standard Deviation — No-PCA vs With-PCA

Inference: The standard deviation comparison reveals that PCA produced more stable predictions (lower variance) for Naïve Bayes and XGBoost, while slightly increasing variance for KNN, Decision Tree, and Gradient Boosting. Logistic Regression and Random Forest showed no meaningful change in stability, indicating that their performance consistency is independent of the feature space dimensionality.

12 Discussion

12.1 Observation Questions

1. Which models improved most with PCA? Which did not? Why?

AdaBoost was the only model that showed a clear accuracy improvement with PCA (+0.88 percentage points on the test set). Its ROC-AUC also improved from 0.9818 to 0.9931. This improvement occurred because PCA removed noisy, redundant features that were causing the weak learners to overfit to irrelevant patterns. In contrast, SVM showed the largest decrease (−1.75 pp) because its optimal linear decision boundary in the full 30-dimensional space was slightly less effective in the 10-component PCA space. Decision Tree and Random Forest also declined, as these models rely on individual feature splits and PCA’s linear combinations obscured the original axis-aligned thresholds they depend on.

2. Did PCA reduce variance across folds (more stable results)?

PCA reduced fold-to-fold variance for Naïve Bayes (std: 0.0287 → 0.0204) and XGBoost (std: 0.0179 → 0.0155). For Naïve Bayes, this occurred because PCA produces uncorrelated components, which better satisfies the feature independence assumption and leads to more consistent predictions. However, several models showed slightly increased variance with PCA (KNN, Decision Tree, AdaBoost, Gradient Boosting), suggesting that the reduced feature space introduced additional sensitivity to fold composition for these algorithms.

3. For high-dimensional data, was PCA beneficial in reducing overfitting?

With 30 features and 569 samples, the WDBC dataset is moderately high-dimensional. PCA compressed the feature space from 30 to 10 components while retaining 95.27% of variance. The cross-validation averages show that Decision Tree (93.19% → 93.85%) and Stacking (97.14% → 97.58%) had higher CV accuracy with PCA, suggesting reduced overfitting for these models. However, the overall impact on overfitting was modest, likely because the dataset does not have an extreme dimensionality-to-sample-size ratio.

4. How did linear models behave compared to ensemble models with PCA?

Logistic Regression showed perfect stability: identical performance (97.37% accuracy, 0.9793 F1) under both settings, indicating it is robust to the dimensionality change at this variance threshold. SVM with a linear kernel decreased slightly (−1.75 pp). Among ensemble models, Gradient Boosting and XGBoost remained unchanged in test accuracy, while Random Forest decreased (−1.75 pp). The key distinction is that linear models benefit from PCA’s removal of multicollinearity (which can destabilize coefficient estimates), while tree-based ensembles are naturally robust to multicollinearity but may lose useful axis-aligned splitting information after PCA’s linear transformation.

5. Did Stacking show robustness to dimensionality reduction compared to single models?

Stacking demonstrated moderate robustness, with a test accuracy change of only -0.88 pp ($96.49\% \rightarrow 95.61\%$) and a CV accuracy improvement of $+0.44$ pp ($97.14\% \rightarrow 97.58\%$). The meta-learner was able to compensate when individual base learners lost performance, because different base models responded differently to PCA. For comparison, the average single-model accuracy change was -0.61 pp, indicating that Stacking's performance shift was comparable to the average individual model.

13 Conclusion

Ten machine learning classifiers were trained and evaluated on the Wisconsin Diagnostic Breast Cancer dataset under two conditions: using all 30 original standardized features (No-PCA) and using 10 PCA components retaining 95.27% of variance (With-PCA). Hyperparameter tuning with 5-fold stratified cross-validation was performed for every model under both settings.

The highest No-PCA test accuracy was achieved by **SVM** at **98.25%**, while **Logistic Regression** achieved the highest With-PCA test accuracy at **97.37%**. Logistic Regression also demonstrated the best cross-validation consistency (CV accuracy: 98.24%) under both conditions.

The results show that PCA's impact is model-dependent: it improved AdaBoost (by removing noisy features), left Logistic Regression, Gradient Boosting, and XGBoost essentially unchanged, and slightly reduced performance for SVM, Decision Tree, and Random Forest. This confirms that PCA is most beneficial when the model is sensitive to multicollinearity or high-dimensional noise, and less impactful for models with built-in feature selection or regularization mechanisms.

14 References

- [1] Scikit-learn Documentation. *Decomposition: PCA*. <https://scikit-learn.org/stable/modules/decomposition.html#pca>
- [2] Scikit-learn Documentation. *Ensemble methods*. <https://scikit-learn.org/stable/modules/ensemble.html>
- [3] Scikit-learn Documentation. *Breast Cancer Wisconsin (Diagnostic) Dataset*. https://scikit-learn.org/stable/datasets/toy_dataset.html#breast-cancer-dataset
- [4] Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer Series in Statistics, 2nd Ed.
- [5] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD Conference.